Improve the Design Workflow of Hardware Engineers Using a Textual DSL With Immediate Graphical Feedback

Twan Bolwerk[1,2(✉)], Marco Alonso[2], and Mathijs Schuts[1,2]

[1] Philips, Best, The Netherlands
{twan.bolwerk,mathijs.schuts}@philips.com
[2] Radboud University, Nijmegen, The Netherlands
marco.alonso@philips.com

Abstract. Cyber-Physical Systems are designed and developed using multi-disciplinary teams that require handovers from one discipline to another. These handovers often involve text documents written in natural language, which can be imprecise, ambiguous, and lead to errors. To address this issue, we created a textual Domain Specific Language with immediate graphical feedback. This language allows mechanical and mechatronic engineers to communicate more effectively during handovers by providing a formalized system description that can be easily visualized in real-time. Our approach also incorporates multiple industry standards, which enables bi-directional navigation between languages, making it easier for teams to collaborate across different disciplines and prevent errors from being made. We applied and evaluated our approach in relation to medical robots at Philips IGT.

Keywords: Domain Specific Language · DSL · Industry Case · Multi-disciplinary · Graphical Feedback · Medical Robots

1 Introduction

A Cyber-Physical System (CPS) [4] is a complex system composed of both hardware and software components. These systems are designed and developed with multi-disciplinary teams. Often, a CPS consists of moving parts such as in the case of robots, cars, airplanes, etc. For the hardware component development, mechanical and mechatronics engineers are involved. The mechanical engineer creates 3D models of the physical components using a Computer Aided Design (CAD) tool [38] and performs measurements, i.e., on weight and tolerances. The mechatronics engineer makes these physical systems move by creating control solutions using tools such as Matlab and Simulink [26]. Both disciplines use their own specialized software tools. Currently, the workflow involves manually written documents that are used to handover designs and measurement information from the mechanical engineer to the mechatronics engineer. Due to the informal nature of these documents, they can be imprecise, ambiguous, and prone to errors. Additionally, changes between document versions may go unnoticed.

G. Kardas et al. (Eds.): FEDCSIS-S3E 2024/KKIO 2024, LNBIP 542, pp. 167–205, 2025.
https://doi.org/10.1007/978-3-031-84913-8_7

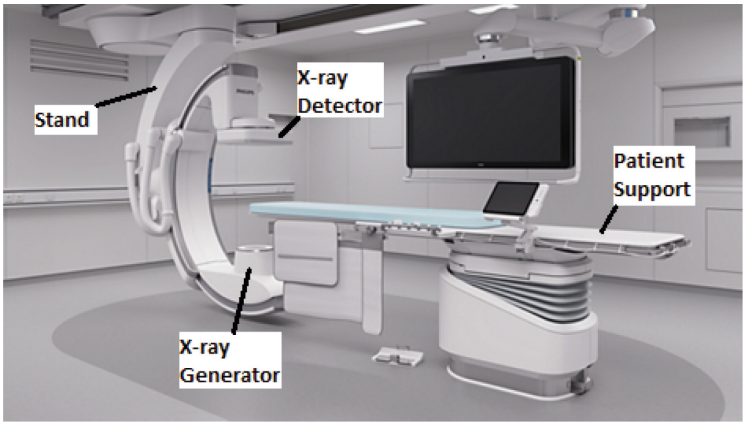

Fig. 1. Interventional X-ray system

At Philips IGT, we create interventional X-ray systems such as the Azurion system in Fig. 1, which are used for minimally invasive procedures. These large medical robots feature motorized moving parts that can be operated using joysticks [35].

In this paper, we present an improved workflow for the development of these CPSs. After a mechanical component has been modelled using a CAD tool, the mechanical engineer can export the 3D model in the Unified Robot Description Format (URDF) [21], which is based on eXtensible Markup Language (XML) [8]. However, one downside of URDF is that weights and tolerances are not included, and cannot be added. Additionally, it lacks an import mechanism for reusing components across similar robots. These limitations can be addressed using the XML macro language (Xacro) [2], which requires manual editing to add weights and tolerances. Both formats are in XML, which is not a user-friendly way of editing. Furthermore, we place these files in a version controlled system, but merging XML-based files is challenging.

This paper introduces an improved workflow and tool enhancements aimed at overcoming these challenges. We created a textual Domain Specific Language (DSL) [12] called Geometry Specific Language (GSL) or GeometrySL. The language extends Xacro but is not based on XML. Instances of this language are placed in a version controlled system and handed over from mechanical engineers to mechatronics engineers. By using formal GSL files instead of informal documents, we reduce the likelihood of errors due to handovers. The GSL provides immediate live graphical feedback when editing the textual instance, showing which part is being edited within the robot's 3D model. It also has facilities for graphically comparing two versions of a robot, highlighting parts that are different in a 3D model. Additionally, it supports bi-directional navigation from a graphical part to the corresponding DSL fragments and vice versa. This allows

seamless navigation between graphical views, URDF instances, Xacro instances, GSL instances, and back again using shortcuts.

To the best of our knowledge, the novelty of this research lies in the creation of the GSL, a DSL that defines how differences between robot representations are visualized. By leveraging multiple languages, including industry standards like Xacro and URDF, this approach enables bi-directional navigation and offers a unique method for visualizing differences in robot descriptions. This paper is an extended version of a conference paper [7]. Compared to the conference version, we present new capabilities in the tool, including a graphical side-by-side view of robot models.

In addition, a detailed evaluation of the new workflow has been conducted with mechatronic engineers at Philips IGT. This evaluation provides insights into how the tool performs in real-world applications and its impact on productivity and collaboration.

This version of the paper discusses the research conducted to establish an effective evaluation framework. By reviewing related work on tool and workflow assessment, we used a combination of existing methodologies for evaluating the new tool.

Also comprehensive comparison of the old and new workflows is presented. This discussion highlights the advantages of the new approach, including reduced reliance on XML editing, improved usability, and better support for version control systems.

By addressing these aspects, this paper provides an overview of the new workflow and its impact on CPS development. The proposed improvements aim to reduce engineering bottlenecks, enhance collaboration, and enable a more intuitive design process for large medical robots.

The paper is organized as follows. In Sect. 2, we provide an overview of related work on technology. We describe the current and proposed workflows in more detail in Sect. 3. The GSL, Xacro and URDF languages are presented in Sect. 4. Section 5 describes the design of the tool. The resulting tool is shown in Sect. 6. In Sect. 7, we describe our evaluation approach and in Sect. 8, we evaluate our tool. Discussion is in Sect. 9. In this section, we also discuss how our work is related to the work of others. And we conclude our paper in Sect. 10.

2 Related Work

Shen et al. [37] categorized recent studies on DSL based on three concerns: concrete syntax, abstract syntax and semantics. They analyzed the parsing and mapping strategies of these studies to classify them into categories such as external/internal, textual/graphical, modeling/visualizing/embedding. This study aims to address research gaps in DSL categorization. The vertical axis of Fig. 2 lists literature references while horizontal axis list the following categories:

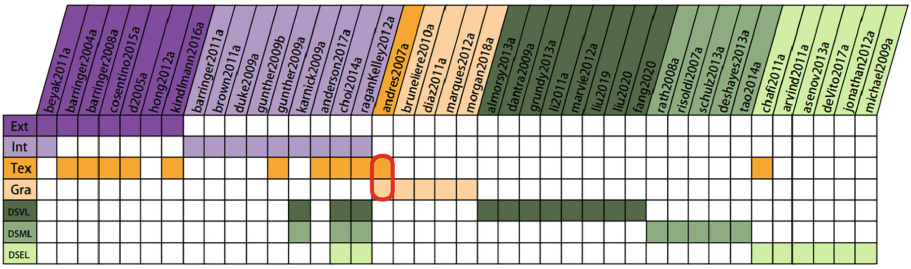

Fig. 2. DSL categories from [37].

- **External (Ext):** Standalone languages with their own syntax and grammar, distinct from any host language, providing specific solutions within a particular domain.
- **Internal (Int):** Embedded within an existing general-purpose programming language, leveraging the host language's syntax and features to implement domain specific constructs.
- **Textual (Tex):** Use text-based syntax, similar to traditional programming languages, designed for domain experts familiar with coding or scripting.
- **Graphical (Gra):** Employ visual representations such as diagrams and flowcharts to define domain specific constructs, useful for users preferring visual over textual representation.
- **Domain Specific Visual Language (DSVL):** A subset of graphical DSLs using specialized visual notations to represent domain concepts, facilitating understanding and communication among stakeholders.
- **Domain Specific Modeling Language (DSML):** Focus on creating models specific to a domain using specialized syntax and semantics, providing tools for simulation, validation, and code generation.
- **Domain Specific Embedded Language (DSEL):** A type of internal DSL embedding domain specific constructs within a host language, integrating domain specific functionality directly into general-purpose language code.

The figure features a red box highlighting the scarcity of research focused on Gra and Tex DSL combinations. While enabling live graphical feedback reduces cognitive load, improves collaboration and communication in engineering projects. This study bridges computing science and behavioral research domains to improve DSL design with live graphical feedback for better communication and collaboration in complex engineering projects.

To visualize property changes in language instances, we need to consider their visibility. Munzner's book [24] recommends automatic highlighting with varied colors, shapes, or positions can emphasize distinctions between properties. The "pop-out" effect in Munzner's book helps users spot differences quickly without focused attention.

In [16], Joshua Horowitz et al. define programming qualities. The focus is on immediate feedback (liveness), domain specific editing (richness) and com-

posability. Composability enables the inclusion of external libraries or components, separating responsibilities over multiple sources. Programming tasks often require using multiple composed tools, so effects should be visible with minimal distraction and effort. Liveness and richness often fail to retain their composability according to Horowitz et al. conclusion. They identified a trend where interfaces lacking composability are standalone applications that offer limited utility in practice. This research explores the intersection of liveness, richness and composability by adhering to these qualities using familiarity with 3D graphical tools for hardware-engineers' workflow improvement and tool intuitiveness enhancement.

Van Rozen et al. [33] recognized the need for program execution observation in traditional programming, which requires re-execution of updated source code from the beginning. This process is time-consuming and distracting when valuable states are lost or difficult to reproduce. To address these challenges, they propose a more fluid and live experience in programming using Textual Model Diff (TMDiff) [30]. Tmdiff uses two key techniques: origin tracking (tracing semantic model elements back to their defining source code) and text differencing (identifying corresponding model elements when aligned names have the same origin). The deltas found by TMDiff are converted into run-time edit operations, which can be applied atomically using rmpatch. Custom state migrations extend rmpatch to avoid information loss or invalid run-time states. Events like user interactions and changes in source code are recorded for undo functionality, persistent application state and back-in-time debugging. They evaluated existing methods (Xtext and EMFCompare) using Eclipse Modeling Framework (EMF) and found TMDiff's scope-handling ability more flexible. Their goal is to minimize distractions and preserve intermediate visual state for a smoother programming experience.

In [11], Cooper et al. present requirements and challenges to integrate graphical editors using Sirius within EMF. They note that while Sirius allows creation of custom modeling editors, it has a steep learning curve. To address this limitation, they propose five requirements for a hybrid textual-graphical workbench: syntax-aware editing, scoping and referencing, rename refactoring, error/warning marker display and accessibility to the textual model. These requirements aim to improve productivity, reduce errors and facilitate collaboration by providing seamless integration between graphical and textual models. Their proposed solution aligns with a case study on hybrid modelling workbenches.

A DSML for UML profiles was created by combining Papyrus and Xtext using EMF. One unique feature is shared storage base for both textual and graphical views, reducing synchronization efforts. The tool is tested their by four scenarios (Create1, Modify1, Create2, Modify2) with experienced developers in the UML language. Results showed that creating elements and setting properties were faster in textual notation while constructing state machines was quicker in the graphical view. Renaming operations were faster in textual mode due to regex search and replace efficiency. The hybrid solution was superior in efficiency, doubling the speed in mentioned scenarios. This demonstrates potential of combined graphical-textual approach for DSMLs [1].

In game development, rapid adjustments of rules are made using tools like Machinations[1]. Van Rozen et al. created a DSL called Micro-Machinations (MM) using the tool Rascal [29] to balance games. MM allows for direct visualization model editing, shortening feedback loops and reducing design iteration times by improving flexibility and adaptability. The Rascal Language WorkBench (LWB) with SPIN model checker [5] is used for analyzing MM, providing an IDE that reads textual MM and displays a visual model interactively. The MM Lib is embedded in the game itself to tackle interoperability, traceability and debugging challenges [41]. An immediate feedback loop greatly improves multi-disciplinary team collaboration in gaming domain similar to engineering domains.

Perez et al. [27] created DSVL using both textual and graphical views with AToM tool[2]. They followed meta-model centric approach where EBNF grammar was generated based on the meta-model, allowing decision to be made later whether to use graphical or textual syntax. Another issue is that produced Abstract Syntax Tree (AST) from parsing is not formally defined causing problems in integration with multi-view DSL proposed by them. They noticed that it is more natural to describe equations in a textual notation.

A more extensive version of this work can be found in [6].

3 Workflow

In this section, we describe the current workflow of hardware engineers at Philips IGT and we propose a new improved workflow. The workflow involves two actors: mechanical engineers and mechatronics engineers.

3.1 Current Workflow

The workflow process follows a waterfall approach, where mechanical engineers measure the system in the factory. These measurements are then communicated via various Office tools to mechatronics engineers. The tools used by these actors are depicted in Fig. 3 and illustrate their interactions. The interactions between the tools can occur either automatically, with data being stored or transferred automatically between tools, or manually inputted by the user.

We describe the actions per actor:

- Mechanical engineer. The mechanical engineer creates the hardware design of the system using the CREO[3] tool for opening and editing 3D CAD files. They also measure the real-world properties of the system, perform calculations in Excel and document any changes compared to the original CAD model using Word. These updates are presented via PowerPoint [36].
- Mechatronic engineer. The mechatronic engineer manually compares previous measurements saved as Matlab instances with the changed properties

[1] https://machinations.io/.
[2] http://atom3.cs.mcgill.ca/.
[3] https://www.ptc.com/en/products/creo/.

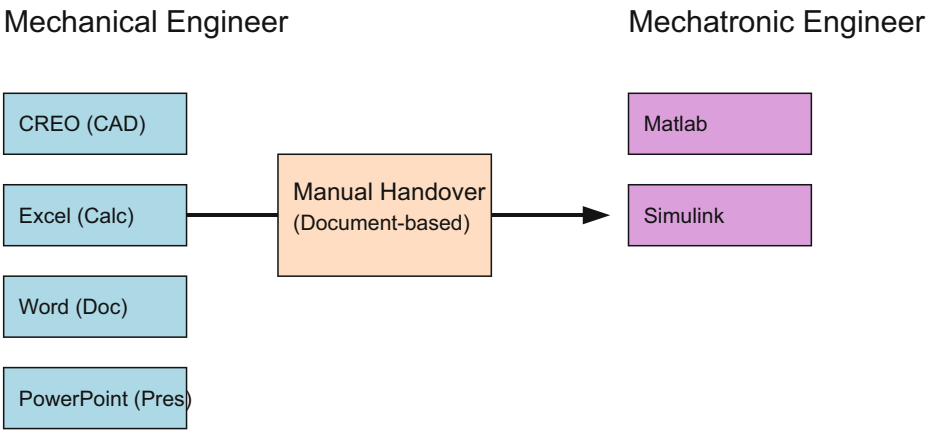

Fig. 3. Tools and actors

recorded in CAD and Excel to determine if the system still meets the specifications. For example, they ensure that the system adheres to the predefined tolerances for each link[4]. These assessments are crucial for maintaining the system's performance and reliability, and are calculated using complex calculations and simulations in Matlab and Simulink [45].

In sum, the handover from the mechanical engineer to the mechatronics engineer currently relies on informal document-based communication. This can result in misunderstandings and potential errors due to missed changes or ambiguities.

3.2 Proposed Workflow

In the proposed workflow, all Microsoft Office tools are replaced by Domain Specific Modelling (DSM) [13] using a single Geometry Specification Language (GSL) with live graphical feedback. It replaces the Microsoft Office tools by a unified language capable of presenting changes, serving as documentation and evaluating mathematical expressions that can be used to describe and calculate properties.

By eliminating the reliance on Microsoft Office tools and therefore multiple sources of truth, the complexity associated with using these tools is reduced. This can be observed in Fig. 4 compared to Fig. 3.

A CAD file can be converted into URDF which we are able to translate to our own DSL. The mechanical engineer can input measurements manually in an expressive manner similar to an Excel spreadsheet but now in the GSL. Meanwhile, the mechatronic engineer can compare previous measurements stored in the single source of truth DSL which is stored within Philips' version control system. Using a textual DSL makes, i.e., merging branches easy.

[4] A link in robotics refers to a rigid component that forms part of a robot's structure, connecting to other links through joints.

3.3 Use Cases

In this section, we describe two main use cases. One involves presenting the textual DSL in a graphical manner and another focuses on visualizing differences between two versions of the DSL in a graphical representation. Differences between robot models can be visualized either by highlighting specific changes or through a side-by-side comparison.

Present. The first use case demonstrates effective communication of measurement changes between mechanical and mechatronic engineers. It serves as a visual representation tool, enabling the presentation of changes through visualization. By hovering over the textual representation using the cursor as a pointer, the view automatically focuses on the specific physical link of the robot being hovered over. This visualization feature enhances communication by clearly highlighting and displaying the changes made to each physical link of the robot. It provides a seamless and intuitive way for stakeholders to understand and interpret updated measurements for every physical link in the robot.

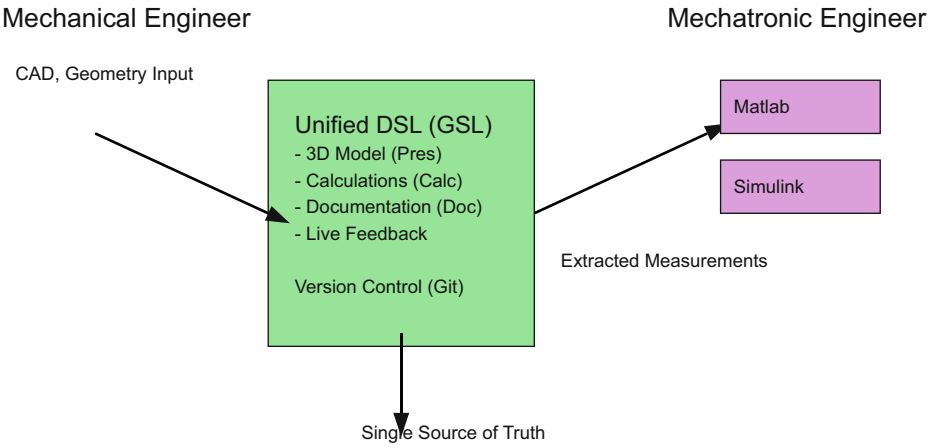

Fig. 4. Actors and tools workflow with DSL.

The mechanical engineer can efficiently navigate, inspect, and present the robot's hardware properties using the tool's textual and graphical representations. This use case showcases how engineers can leverage the tool's features to communicate and demonstrate the robot's hardware-properties.

Highlight. The highlight use case offers an efficient solution for mechatronic engineers to automatically identify and highlight changed physical links between robot or component versions. This internal scenario eliminates the need for manual input of values into Matlab, as it leverages automatic highlighting and live graphical feedback. Engineers can quickly and accurately pinpoint variations

between link versions using this feature, reducing reliance on manual comparisons and potential errors.

Side-by-Side. The side-by-side view provides users with greater control by allowing them to visually detect changes such as variations in the color and size of links. This approach makes differences graphically more apparent compared to highlighting, which merely indicates where a change has occurred without offering a clear visual representation.

4 Languages

In this section, we introduce the Language WorkBench (LWB) used to implement our tool as needed in order to understand this paper. We describe the URDF, Xacro and GSL languages. Finally, we provide example instances of these three languages.

4.1 Language Workbench

A LWB provide concepts and mechanisms to define language syntax, semantics, and code generation for a language. They facilitate model-driven engineering by allowing developers and domain experts to work with high-level abstractions that closely resemble the specific problem domain. This can lead to more efficient development processes, maintainable code, and closer collaboration between different stakeholders. LWBs bridge the gap between generalized programming languages and the unique requirements of specialized domains [9].

There are many language workbenches (LWBs) available. Both Eclipse Modeling Framework (EMF) and Rascal[5] are used at Philips; however, Rascal provides support for Visual Studio Code (VS Code). Given that VS Code[6] is the preferred tool at Philips and aligns with our previous experience using Rascal, we decided to use it for our tool.

Rascal [19] is a type-safe programming language featuring immutable data, built-in pattern matching, search, and relational calculus. It is a functional and procedural programming language with Java-like syntax. We introduce the language in this section as needed to understand the code fragments presented in this paper. The code snippets in Listing 1.1 are reused from [34].

```
1 loc l =
2   |file:///Users/kees/.bashrc|(100,20,<2,0>,<2,20>)
3
4 data Boolean = true() | false() | and(Boolean lhs,
5   Boolean rhs);
6 //extending Boolean with another constructor
7 data Boolean = or(Boolean lhs, Boolean rhs);
```

[5] https://www.rascal-mpl.org.
[6] https://code.visualstudio.com.

```
8 data Statement = \if(Expression c, Statement tt,
9   Statement ff);
10
11 for (int i <- [1 .. 5]) println(i);
12
13 str w = "world"
14 println("Hello, <w>!");          //prints "Hello, world!"
```

Listing 1.1. Rascal code fragments

Rascal has numerical types such as `int` and booleans, represented by `bool`. It supports polymorphic `lists`, `maps` for collections and `loc` for location constants.

Rascal also provides the following built-in functions for working with `maps`:

1. Map := A list that uses any kind of data as index (called keys), to store a value.
2. rangeR := Expects a map and returns a map of key, value pairs that match the values.
3. range := Returns a list of values.
4. domain := Returns a list of keys.

On Line 2 there is a constant `loc` that points to a file with the file scheme and selects the part on line 2 between the left margin and the 20^{th} column. Locations are used to refer to files, store information extracted from files and help in referring back to source locations.

In Rascal, Algebraic Data Types (ADTs) can be user-defined with their constructor functions. The fragment on Lines 4–9 shows a declaration of an ADT for the representation of Boolean expressions using three constructors. The next line extends the same ADT by adding an alternative to the existing declaration. Reserved keywords are not permitted as names of algebraic constructors; hence, `if` is escaped with `\if` when used as the name of an algebraic constructor.

Control structures such as `for` can be used to iterate over a value, while `str` literals in Rascal are not delimited by line endings.

We utilized the following two Rascal libraries: Salix and TypePal. Salix is a library that facilitates the development of web-based GUI programs; it runs user code on the server side instead of client-side execution. The library employs the Model View Controller (MVC) pattern by sending HTML patches to the browser and interpreting messages from the browser on the server to update the view accordingly. TypePal is a typechecking and validator library for Rascal, designed to analyze and enforce type constraints, ensuring correct usage of types and detecting potential type-related errors in DSL instances. TypePal provides static type checking capabilities and can be used to improve the reliability and correctness of language instances [40].

4.2 URDF

Utilizing URDF offers numerous advantages, notably its capability to express a wide range of hardware properties, with the exception of tolerances. Moreover,

URDF facilitates seamless conversion from formats like CAD files, streamlining the integration of engineering disciplines for our use case. Furthermore, the compatibility of URDF with visualization tools such as RViz[7] underscores its ability in conveying essential information using a 3D graphical representation.

The downside of URDF is its inability to scale and its cumbersome XML format, which is difficult to maintain and causes issues in the archiving system when merging changes. As the model becomes more complex, the lack of reusable components results in larger file sizes.

4.3 Xacro

To address the issue of large URDF files, especially for complex 3D robot models like interventional X-ray systems, a solution based on modularization and composition using the Xacro language can be employed. Xacro provides a way to create modular and reusable components, making robot descriptions more manageable and organized.

Although Xacro simplifies URDF composition and introduces expression evaluation, it still relies on an XML format that is neither user-friendly nor intuitive, making it difficult to version control and prone to merging issues.

4.4 GeometrySL

To overcome the previously described limitations, we introduce GeometrySL (GSL), which extends XacroSL. GeometrySL encapsulates Xacro and converts it to a more user-friendly syntax, making it easier to archive and merge changes. It enabled the creation of a more intuitive syntax that supports custom property definitions (including tolerances) and provides enhanced visualization options. The aim is to offer a more intuitive, flexible language for 3D robot modeling, addressing the drawbacks associated with traditional XML-based URDF descriptions.

Creating GeometrySL adds flexibility in defining and designing custom syntax, making it more intuitive to use and easier to extend with new semantics and introduce custom properties, enhancing its expressiveness and adaptability for different use cases.

One of these new use cases is the integration of visual semantics. This allows mechanical engineers to describe desired appearances of views using a language that engineers can understand. This approach enables better collaboration by providing live graphical feedback.

4.5 Example Instances

Next, we will describe three features using language instances–custom features, modular includes, and highlighting differences between two robot versions. Due

[7] http://wiki.ros.org/rviz.

to confidentiality concerns, we use Franka's Panda robot[8] as an example instead of our interventional X-ray system.

```
robot {
  link {
    name="lbr_iiwa_link_0"
    inertial {
      origin := {
        rpy="0 0 0"
        xyz="-0.1 0 0.07"
      }
      mass := { value="0.2" }
      tolerance := { value="200" }
      inertia := {
        ixx="0.05"
        ixy="1"
        ixz="0"
        iyy="0.06"
        iyz="0"
        izz="0.03"
      }
    }
    visual {
      origin := {
        rpy="0 0 0"
        xyz="0.2 0.1 0"
      }
      geometry {
        mesh := { filename="meshes/link_0.stl" }
      }
    }
    collision {
      origin := {
        rpy="0 0 0"
        xyz="0 0 0"
      }
      geometry {
        mesh := { filename="meshes/link_0.stl" }
      }
    }
  }
}
```

Listing 1.2. GeometrySL instance example

Listing 1.2 shows an example of GeometrySL for the Panda robot, demonstrating how to define a link. A link has a name and various features such as inertial, visual, and collision properties. Inside the inertial feature, we added a custom property called "tolerance". This property can be exported to Xacro

[8] https://support.franka.de/docs/franka_ros.html.

but not to URDF. The instance also references STereo Lithography (STL) files, which is a format used to describe 3D objects using triangles.

```
robot {
  name="lbr_iiwa"
  xmlns:xacro="http://www.ros.org/wiki/xacro"
  include "lbr_iiwa_link_0.gsl"
  ...
  include "lbr_iiwa_link_7.gsl"

  include "lbr_iiwa_joint_1.gsl"
  ...
  include "lbr_iiwa_joint_7.gsl"
}
```

Listing 1.3. GeometrySL instance example for modular includes

In Listing 1.3, an instance of GeometrySL is shown that describes the Panda robot. It includes other instances of GeometrySL to describe links and joints of the Panda robot. A link is represented as a mesh, while a joint defines the relation between exactly two joints.

```
<?xml version="1.0"?>
<robot name ="lbr_iiwa"
  xmlns:xacro="http://www.ros.org/wiki/xacro" >
<xacro:property name ="color" value ="Green"/>
<xacro:property name ="half" value ="0.1"/>
<xacro:include filename
  ="lbr_iiwa_link_0.gsl.xacro"/>
...
<xacro:include filename
  ="lbr_iiwa_link_7.gsl.xacro"/>

<xacro:include filename
  ="lbr_iiwa_joint_1.gsl.xacro"/>
...
<xacro:include filename
  ="lbr_iiwa_joint_7.gsl.xacro"/>
</robot>
```

Listing 1.4. Xacro instance example for modular includes

Listing 1.4 shows an example of how to represent the same information from Listing 1.3 in the Xacro language. It uses XML syntax instead.

```
<?xml version="1.0" ?>
<robot name="lbr_iiwa">
  <link name="lbr_iiwa_link_0">
    <inertial>
      <origin rpy="0 0 0" xyz="-0.1 0 0.07"/>
      <mass value="0.2"/>
      <inertia ixx="0.05" ixy="1" ixz="0"
```

```
        iyy="0.06" iyz="0" izz="0.03"/>
    </inertial>
    <visual>
      <origin rpy="0 0 0" xyz="0.2 0.1 0"/>
      <geometry>
        <mesh filename="meshes/link_0.stl"/>
      </geometry>
      <material name="Grey"/>
    </visual>
    <collision>
      <origin rpy="0 0 0" xyz="0 0 0"/>
      <geometry>
        <mesh filename="meshes/link_0.stl"/>
      </geometry>
    </collision>
  </link>
  ...
  <link name="lbr_iiwa_link_7">
    ...
  </link>
  <joint name="lbr_iiwa_joint_1" type="revolute">
    <parent link="lbr_iiwa_link_0"/>
    <child link="lbr_iiwa_link_1"/>
    <origin rpy="0 0 0" xyz="0 0 0.1575"/>
    <axis xyz="1 0 1"/>
    <limit effort="300" lower="-2.96705972839"
      upper="2.96705972839" velocity="10"/>
    <dynamics damping="0.5"/>
  </joint>
  ...
  <joint name="lbr_iiwa_joint_7" type="revolute">
    ...
  </joint>
</robot>
```

Listing 1.5. URDF instance example for expanded Xacro instance

Listing 1.5 shows an example of how to represent the same information as Listing 1.4 using URDF syntax instead. It includes expanded information from Listing 1.2, but does not include the "tolerance" property that was present in GeometrySL and Xacro.

```
highlight
  robot "robot_v1.gsl"
difference
  robot "robot_v2.gsl"
```

Listing 1.6. GeometrySL instance example for highlighting differences for two versions of a robot

Listing 1.6 demonstrates a language instance of GeometrySL that visually shows the differences between two versions of the robot.

```
compare
  robot "robot_v1.gsl"
with
  robot "robot_v2.gsl"
```

Listing 1.7. GeometrySL instance example for compare with for two versions of a robot

Listing 1.7 shows a language instance of GeometrySL that will show both robot instances side-by-side.

5 Design

In this section, we describe how we realized the use cases presented in Sect. 3.

5.1 Conversion Tool

One of the benefits of using URDF, a widely adopted and standardized format is that other tools often have conversion features. For instance, the Blender tool[9] can be utilized as an intermediary that enables conversion from CAD export files into URDF. Blender is open-source, has free licensing, and it has widespread community support. This not only makes Blender cost-effective but also ensures that the tool is consistently updated and improved by a global community of developers.

5.2 VS Code URDF Viewer

The VS Code URDF viewer[10] was utilized as the starting point, which incorporates BabylonJS[11], a JavaScript 3D graphics library. This library can load STL files and assemble them using URDF. However, it lacks certain features such as comparing changed properties, highlighting differences, bi-directional navigation and side-by-side views, and maintaining state after changes. Additionally, it requires the use of URDF, which has a non-user-friendly syntax and leads to large file sizes when designing systems like interventional X-ray systems. Nonetheless, despite these limitations, the VS Code URDF viewer serves as a beneficial starting point.

The visualization is extended with a context menu. The view's context menu enhances user experience by enabling the identification of links through right-click actions, revealing a menu that displays the link's name, depicted in Fig. 6. This visual representation establishes a direct connection between the textual and visual physical links of the robot, improving cohesiveness and comprehension of both representations.

The sliders facilitate joint movement, enhancing the view by providing an interactive experience. While primarily focused on static properties, this feature

[9] https://www.blender.org/.
[10] https://github.com/javahacks/vscode-urdf-viewer.
[11] https://www.babylonjs.com/.

can greatly improve the visualization of specific links. Furthermore, the “reload” and “auto” buttons play a vital role in updating the view with the latest changes made in the GSL textual editor, whether through manual input or automatic updates.

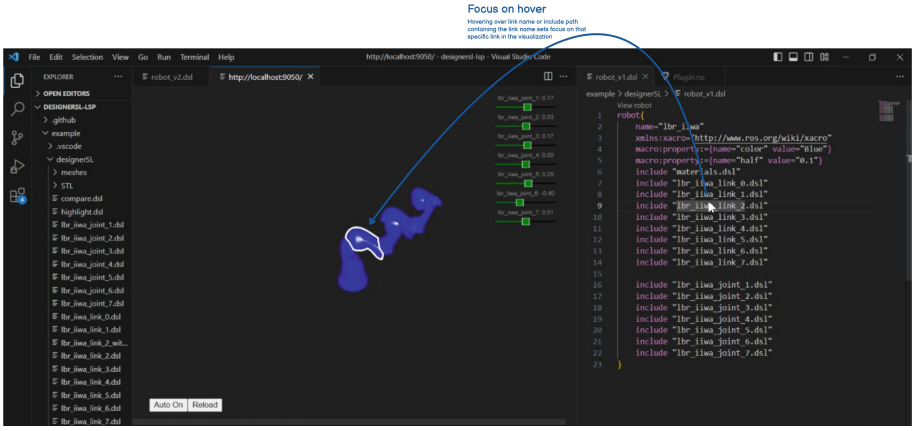

Fig. 5. Focus on hover link

5.3 Show Differences Between Robot Versions

The next use case is showing differences between two robot versions. When the GSL instances are defined as the corresponding physical links of the robot are displayed as solid while the remaining links become transparent, offering a visual distinction.

Listing 1.6 provides a GSL instance to compare two robot versions. In Fig. 6, we show the same GSL instance on the right and on the link we have the graphical view with differences.

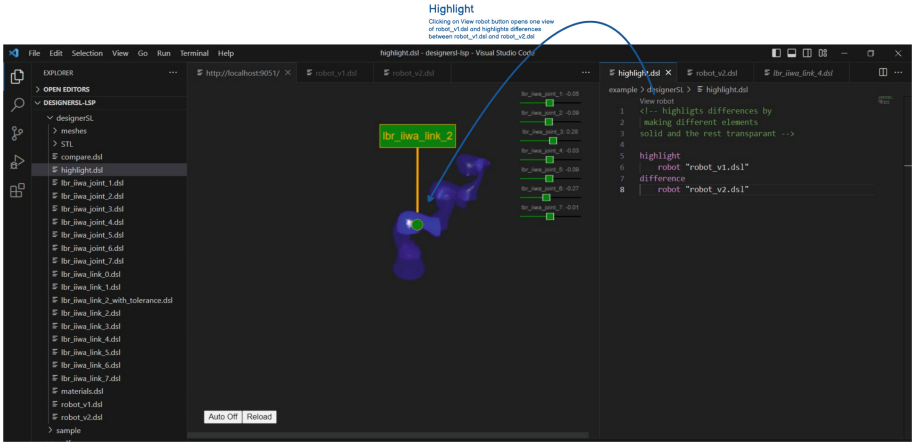

Fig. 6. Highlight robot

5.4 Present Robot

The first use case we are going to describe is how we present the robot. The conversion tool has been executed and the VS Code URDF viewer is running. Salix is utilized to present the robot by a polling feature that performs an action over a specified interval as short as one second. Although this feature may impose certain performance overhead, it helps our language to fulfill the "liveness" quality criteria by consistently checking for differences and updating the view accordingly. A toggle button labeled "auto" is provided, as shown in Fig. 5, which allows users to enable or disable the polling feature. This functionality offers a option to conserve system resources. With polling enabled the automatic updates are enabled, thus allowing for immediate feedback.

Upon detection of any discrepancy, a message with the updated model is send to our viewer which runs on a separate webserver thread. Rather than refreshing the complete web-view, the web-server can update the visualization gradually. This ensures that the robot is always maintained in the view during these gradual updates, reducing distractions from changing visual context. This feature enhances the user experience by providing a seamless transition during updates and maintaining continuity in the visualization.

Each time the user changes the source code and then saves the source code, a Rascal "summarize" event is triggered. In this event we set a flag to true, the polling mechanism that runs on a separate web-server thread then sends a HTTP message to our viewer. The viewer is equipped with a so-called listener (hooks) which update the visualization while preserving state. This significantly reduces distraction of reloading graphics and therefore improving usability.

The "polling" mechanism is used to focus on an element that is hovered over by the mouse. This improves the user's understanding of where the user is in source code. The focus mechanism sets the camera focus on that specific element and marks the element with a specific outline color as shown in Fig. 5.

In this approach, a custom Xacro parser has been developed to convert Xacro code into GeometrySL. The shell "exec" function from Rascal is leveraged to call the Xacro compiler.

5.5 Tool's Components Diagram

The component diagram in Fig. 7 illustrates the software components and their relationships.

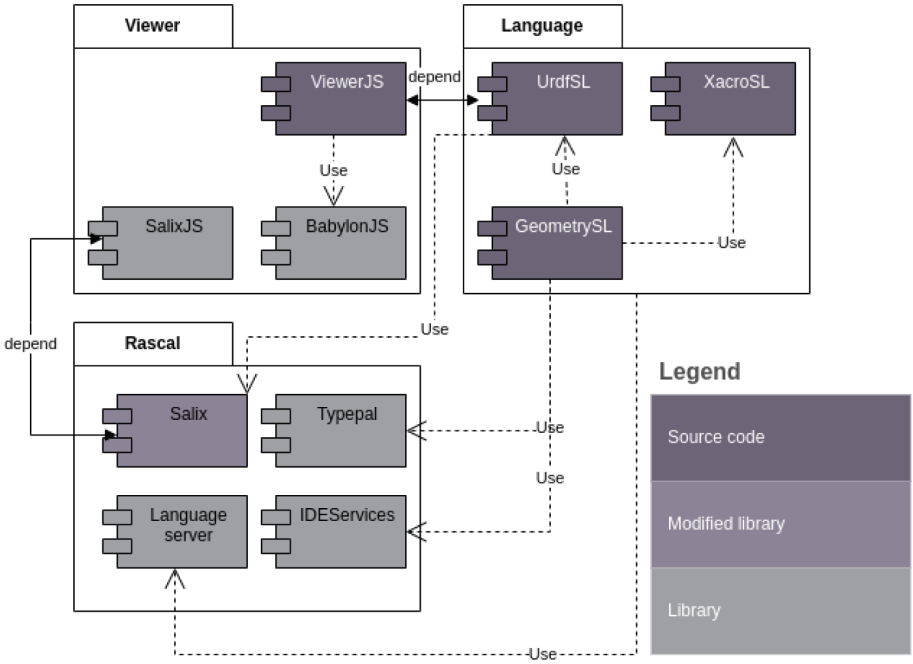

Fig. 7. Components.

Each component shown in Fig. 7 is mapped to its respective responsibility in Table 1.

Table 1. Responsibility per component

Component	Responsibility
ViewerJS and UrdfSL	Visualization of robots and changes
Babylon JS	3D visualization
SalixJS and Salix	Bi-directional navigation
TypePal	Checking path existence and navigation
LanguageServer	Integration with the IDE, using events (summarize, document, lenses)
IDEServices	Opening files in the IDE or opening interactive content
XacroSL	Expression evaluation and resolving include path, using Xacro
GeometrySL	Providing visual semantics and URDF conversion

5.6 Link Origin Tracing

In our work, we trace link origin throughout the transformation process from GeometrySL to Xacro and finally to URDF, and vice versa. This approach draws inspiration from Inostroza et al. [17], where they track string origins during transformations. However, our adaptation involves storing origin locations from physical links of the robot instead of strings and preserving this information across three transformations instead of one. This feature enables seamless navigation between the URDF source, GSL instance, and graphical representation of the robot model. As demonstrated previously, hovering over textual elements triggers the graphical element to be highlighted with a white outline. When holding down the Ctrl-key and clicking on a visual element, the user is directed to the corresponding section in the URDF source code. Conversely, clicking on a visual element without holding down Ctrl-key directs the user to the corresponding section in the GSL instance. With this approach bi-directional navigation is created. Upon hovering over text, the camera dynamically adjusts to center the active link, enhancing visual focus. When clicking on the link, a text editor is triggered, displaying the corresponding element's description for detailed examination. The tracing of robot link elements is chosen because links contain the visually represented graphical mesh, allowing for bi-directional navigation with the graphical representation.

By re-using the shared abstract data structure of Xacro, the compatibility with Xacro is maintained and the development time of GeometrySL is reduced. The shared data structure, illustrated in Listing 1.8, encapsulates both Xacro and GeometrySL and is located in the Shared folder of the class diagram.

```
data Id_ = id(str id);

data XACRO_Attribute = attribute(Id_ \type, str val)
  | xacro_attribute(Id_ \type1,Id_ \type2, str val);

data XACRO_Object = ... // irrelevant alternatives omitted
  | link(Id_ name,
      list[XACRO_Attribute] attributes,
      list[XACRO_Object] elements,
      loc origin=|unknown:///|); // link origin tracing

data XACRO = robot__(list[XACRO_Attribute] attributes,
  list[XACRO_Object] elements,
  loc origin=|unknown:///|); // link origin tracing
```

Listing 1.8. Shared XACRO datastructure

An important point to highlight in Listing 1.8 is that all objects and attributes are generic, allowing for easy extension of new robot property semantics such as tolerances. However, in order to support bi-directional navigation a more concrete data structure is needed. The link is specifically specified to incorporate storing origin locations.

```
tuple[XACRO_Object,
  map[str,XACRO]] translate(Object obj,
    map[str,XACRO] includes){
  <xacro_obj,includes> = translate(obj.\type, obj, includes);
  if (xacro_obj.\type.id == "link") {
    name = get(l.attributes, "name");
    return <link(id(name),
      xacro_obj.attributes,
      xacro_obj.elements,
      origin=obj@\loc), // keep track of link location
      includes>;
  }
  return <xacro_obj, includes>;
}
```

Listing 1.9. GeometrySL::Semantics

By opting for this generic data structure, it accommodates all valid XML formats, enabling the parsing and semantic translation of custom properties. However, a trade-off of this approach is that specific tasks like determining link origins requires iterating through all, as illustrated in Listing 1.11. Furthermore, the semantics of the generic data structure do not verify the validity of elements, accepting all inputs, whereas the UrdfSL semantics, as depicted in the Appendix' Listing 1.14, are more specific and restrictive.

```
map[str,loc] linkLocationMap = ();

map[str, loc] gatherLinkLocation(map[str,XACRO] xacros)
{
  map[str, loc] result = ();
  for (key <- xacros){
    result += gatherLinkLocation(xacros[key]);
  }
  return result;
}

map[str,loc] gatherLinkLocation(XACRO xacro){
  map[str, loc] result = ();
  for (obj <- xacro.elements){
    // pattern match on link elements
    if (link(_, _, _) := obj) {
      // map link name and location
      result += (obj.name.id:obj.origin);
    }
  }
  return result;
}
```

Listing 1.10. Gather link location algorithm

In Listing 1.10 the link location map is a relation that can be used in both directions. We can search on the link name but also look for its location to get the name of the link, a functionality that proves notably convenient.

5.7 Focus on Hover

Different Integrated Development Environment (IDE) events are utilized, specifically employing the documenter event. This event has information about where the cursor of the user is located. This active cursor information is used to determine what link is being inspected. In our implementation we even were able to look up the active link through include statements.

```
bool isCursorLocationInLocation(loc cursor, loc linkLocation){
  return cursor.begin >= linkLocation.begin &&
    cursor.end <= linkLocation.end;
}
```

Listing 1.11. Cursor location algorithm

In the link translation from GeometrySL to URDF we keep track of the identifier, the name of the link and its location see Listing 1.11. This goes in two directions, hence bi-directional. The identifier is used to find the corresponding location in the link location mapping and in the other direction to find the identifier based on its location. This location lookup checks if a certain location is in the same file and in between the row and column. If this is the case, it will return its identifier.

5.8 Bi-directional Navigation

With link-origin-tracing it possible to also use the 3D visualization to navigate through source code. When clicking on a visualized link it opens the corresponding section of code. This can be seen in Fig. 8.

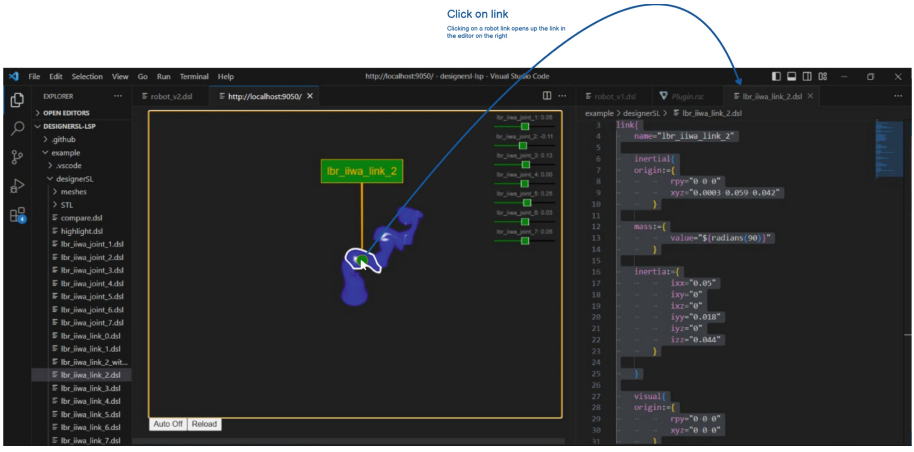

Fig. 8. Click on visual link.

5.9 Highlight Differences

The functionality of highlighting differences compared to previous version(s). Link origin tracing must be ignored, in order to strictly check for value equality, this is different opposed to [30] where origin is actually added and used to ensure file equality. The links are hashed such that they can be compared and stored more efficiently, see *removeOriginFromLink* function Listing 1.12.

```
map[str,str] removeOriginFromLink(list[URDF] links){
  map[str,str] result = ();
  for (link <- links){
    str uniqueHash = md5Hash(link.attributes + link.elements);
    result += (getName(link).val: uniqueHash);
  }
  return result;
}
```

Listing 1.12. Remove origin from link implementation

The URDF data structure consists of concrete data types for each URDF property. Each property has elements and attributes, parsed from the URDF robots. In order to compare robot1 (r1) and robot2 (r2), we extract from the elements mapping the "link" and store these in l1 and l2 respectively, such that we compare links only. Recall the explanation of the build-in `map` functions in Sect. 4.1. With rangeR we exclude the links in robot1 (r1) that do not exists in the robot2 (r2). Next the domain function is applied to the result, such that only the link identifiers are returned, since the keys are the link names, see Listing 1.12.

```
list [str] compare(URDF r1, URDF r2){
    l1 = getAll(r1, "link");
    l2 = getAll(r2, "link");
    return toList(domain(rangeR(
              removeOriginFromLink(l1),
                range(removeOriginFromLink(l2)))));
}
```

Listing 1.13. Compare algorithm

Note that we chose to apply the algorithm to the URDF data structure instead of GeometrySL or XacroSL due to the URDF's single-file nature, simplifying the process. We check for changes with the algorithm in Listing 1.13.

5.10 Compare Side-by-Side Differences

Alternatively to the approach of highlighting differences, we opted for a side-by-side comparison of two different versions of robot description files, as illustrated in Fig. 9. This approach allows mechatronic engineers to simultaneously view and compare the models in a split view, with one version displayed on the left

and the other on the right. Differences in properties, structure, or elements can be visually identified, offering an intuitive and straightforward analysis. Interactive features, such as highlighting elements when hovering over them, further streamline the comparison process. This method leverages live graphical feedback, enabling engineers to efficiently analyze and validate the variations between two robot versions.

Fig. 9. Compare side-by-side.

6 Results

To view the final product in action, we recommend watching the demonstration videos. For confidentiality reasons, we use the Panda robot model instead of the Philips IGT interventional X-ray system.

The first video[12] illustrates two scenarios: presenting the robot and highlighting differences between two versions. The following steps are demonstrated:

1. The user clicks the left mouse button on a physical link of the robot in the graphical view.
2. The tool opens the corresponding `GeometrySL` instance and highlights the relevant text fragment.
3. The user switches to the textual view and opens the top-level `GSL` instance.
4. The user returns to the graphical view, presses the `Ctrl` key, and left-clicks on a link simultaneously.
5. The tool opens the corresponding `URDF` instance and highlights the associated text fragment.

[12] https://www.youtube.com/watch?v=n71kg1OKVus&ab_channel=fedcsis3391.

6. The user navigates back to the textual view and opens the top-level `GSL` instance again.
7. The user `Ctrl`-clicks on an included `GSL` file.
8. The tool opens the selected file.
9. The user navigates to the textual view and reopens the top-level `GSL` instance.
10. The user returns to the graphical view and presses the right mouse button.
11. The tool displays the name of the physical link.
12. The user left-clicks again on a physical link of the robot in the graphical view.
13. The tool opens the corresponding `GeometrySL` instance and highlights the relevant text fragment.

These steps demonstrate how the tool seamlessly integrates graphical and textual views, enabling users to navigate between robot elements and their underlying descriptions. This integration supports efficient analysis and modification workflows.

The second video[13] showcases the hover feature. The following sequence is presented:

1. The user changes the robot description file from version 1 to version 2.
2. The tool automatically updates the graphical view to reflect robot version 2.
3. The user types the keywords **highlight** robot version 1 **difference** robot version 2.
4. The tool graphically highlights the differences between the two robot versions by increasing their opacity.
5. The user navigates to the textual view and opens the top-level `GSL` instance.
6. Using `Ctrl`-click, the user selects an included `GSL` file.
7. The tool opens the selected file.
8. The user hovers over the included robot links.
9. While hovering, the tool highlights the corresponding links in the graphical view with a bold white outline.
10. The user identifies link 6, an **orange** component, in the graphical view and `Ctrl`-clicks on its associated `GSL` file.
11. The user navigates to the property color and changes it from **orange** to **black**.
12. The tool automatically updates the graphical view, changing the bold white outline and the color of link 6 from orange to black.

This sequence demonstrates how the tool integrates hover functionality, graphical feedback, and real-time updates, enhancing user interaction and streamlining the process of analyzing and modifying robot descriptions.

[13] https://www.youtube.com/watch?v=o_bJ8NsEODQ&t=9s&ab_channel=fedcsis3391.

The third video[14] provides a comprehensive overview of all the tool's features in action. It highlights the tool's ability to adjust the rotation of individual robot links using sliders and employs two distinct robot description files, `robot_v1.dsl` and `robot_v2.dsl`, for clarity.

In addition to the scenarios described above, the video demonstrates the side-by-side comparison feature, allowing users to easily observe differences between robot versions. The tool provides full control over how the graphical viewer represents robot descriptions. Users can choose between automatically highlighting differences, displaying the versions side-by-side, or focusing on a single version.

Moreover, the "compare with" and "highlight differences" keywords illustrate how the tool's functionality can be extended with additional keywords, enabling support for a variety of graphical representations. This flexibility and adaptability further enhance the tool's potential for future development and functionality expansion.

7 Evaluation Approach

Various articles on DSL evaluation, such as [20,39], and [18], rely on **questionnaires** to measure DSL usability. Despite the existence of alternative approaches like recording user interactions or analyzing user reactions [23], privacy concerns discourage their use. A **task-based evaluation** approach was chosen by Hoffman et al. [15] to assess the efficiency of the DSL.

Prior to the tasks, the test group received learning materials providing instructions on using the language. After performing the tasks, a questionnaire was used to gather initial impressions and obtain feedback on the learning materials. Participants' programming experience was considered in the result section with regard to the performance measure.

The task-based approach used questions and time as an indicator of efficiency. Answers to validate the accuracy of the users and time to measure the efficiency of each user. By keeping track of these two factors, the authors were able to mathematically measure efficiency; the researchers employed the Rate Correct Score (RCS) formula [44], defined as;

$$\mathrm{RCS} = \frac{c}{\sum \mathrm{RT}} \tag{1}$$

where c represents the number of correct responses and $\sum \mathrm{RT}$ is the total reaction time for a trial [42]. This formula originates from the behavioral science field, and many adaptations of it exist, such as RCS.

Points Per Minute (PPM) introduced by Hoffmann et al. [15], similar to another measure called throughput [28], reflects correct responses per qualified time unit. In the case of PPM, the time unit is minute. Higher RCS, PPM, and throughput values suggest faster, more efficient task completion. However, a high

[14] https://www.youtube.com/watch?v=XiwrbvOOKiA&ab_channel=graphicaldesigner.

score in less time might not always surpass a higher overall score achieved over longer periods.

The balance between speed and accuracy should be carefully considered, keeping in mind the task or domain requirements. As noted by Kahraman & Bilgen [18], in addition to questionnaires, which may be highly subjective, the construct validity framework incorporates assessment evidence.

In this evaluation, the evidence could include log files collected during the participant's use of the tool, providing a more objective measure. The RCS and PPM serve as benchmarks for evaluating user efficiency. These benchmarks are utilized to assess future enhancements and determine whether they have effectively improved user efficiency.

8 Evaluation

For the evaluation of DesignerSL at Philips IGT, a preperation deployment is conducted followed by the evaluation of two phases: an alpha test phase and a beta test phase.

8.1 Deployment

In the preliminary evaluation, it was found that the installation process of the language(s) posed challenges for hardware engineers, as it involved installation of multiple programs. The Xacro functionality has been incorporated, and now only Python and the Rascal extension for Visual Studio Code (VS Code) are required.

8.2 Alpha

During the alpha testing phase, it was observed that the execution of import and command-line instructions was not intuitive for mechanical engineers with limited programming background.

To address this challenge, the decision was made to deploy the language as a VS Code extension. This approach involves creating a .vsx executable, simplifying the setup process. By relying on the installation of VS Code, the barriers to tool usage are reduced. As an interim solution, a script has been implemented to execute the command-line instructions with a single click, providing a more user-friendly experience.

Furthermore, the alpha testing revealed that not only links but also joints were being edited.

To enhance clarity, the aim is to visualize joints by highlighting both the parent and child links affected by the changes. This visual representation will provide users with a clearer understanding of the impact of their modifications.

In addition, the current tool converts a URDF to DesignerSL with a single click, but the links and joints are not separated into individual files.

As a future improvement, the workflow will be enhanced by automatically separating the links and joints into separate files. This modification will simplify maintenance and further streamline the transition from URDF to DesignerSL.

8.3 Beta

The beta test group conducted tasks derived from the use cases. To calculate the Points per Minute (PPM) metric, each task was accompanied by a question. The time taken for each task was recorded through the logs of the tool.

Upon completion of the tasks, the participants were asked to fill out a questionnaire about the usability of the language. The participant group comprised 5 mechatronic engineers and 4 software engineers, with the latter excluded from the test results due to their non-inclusion as intended users. The results revealed variations among mechatronic engineers in task completion within the 20-minute timeframe, as seen in Evalutation Results Table 2, which displays each task's completion time in minutes, RCS, and PPM results. The evaluation form and comments were also taken into account.

The following key points provide an in-depth analysis of these outcomes, emphasizing tasks completed consistently and mechatronic users' perceptions of the tool. It's worth emphasizing that mechatronic engineers didn't use the office stack that this new tool now supersedes. As a result, they do not utilize the document, presentation, and calculation use cases, and they do not make comparisons with these specific tasks (1, 2, 3, 4, 5). Their workflow involves notion of changes in measurements. Instead of manually comparing textual variations, they now benefit from visual feedback, which enables them to effortlessly observe changes, make comparisons, and identify and highlight differences, which are then used in Matlab to calculate and verify the performance of the system.

In Table 2, we have listed the mechatronic engineers who took part in the beta evaluation. Each column corresponds to tasks 1 through 6. A task was awarded full points when the result was correct. If a task was only partially correct, it received 0.5 points. Tasks that were not completed or received no response are denoted by a '-' symbol.

Table 2. Evaluation Results

Role	1	2	3	4	5	6	Minutes	RT in seconds	RCS	PPM
Mechatronic 1	1	1	1	1	1	1	15	900	0.006666667	0.4
Mechatronic 2	1	-	-	-	-	-	20	1200	0.000833333	0.05
Mechatronic 3	1	1	-	-	-	-	20	1200	0.001666667	0.1
Mechatronic 4	1	1	1	1	1	0.5	17	1020	0.005882353	0.352941176
Mechatronic 5	1	1	1	1	1	1	20	1200	0.005	0.3
Average PPM: 0.240588235										

1. **Points per Minute (PPM) metric**: To assess the efficiency of the users during task performance, the PPM metric was employed [42]. This calculation depends on the RCS [44]. The evaluation session had a total duration of 40 min, excluding the introduction and feedback round. The evaluation began

with a set of six initial steps designed to familiarize participants with the tool. Subsequently, participants engaged in a task-based approach, which lasted for 20 min and encompassed six tasks.

2. **Environment Impact**: The beta test group conducted the tasks in a conference room using their laptops. However, the lack of additional monitors made it difficult for them to use the tool with split-screen view, especially on smaller screens.
3. **User Interface Observations**: The group also made some observations about the user interface of the software:
   - "It looks really cool and visuals are responsive"
   - "The software does not allow changing the robot position after focusing on hover."
   - "Users prefer automatic updates."
   - "Opening the text editor replaces the view in Visual Studio Code, but this issue was resolved by using a web browser."
   - "It is really cramped up to work with on a single screen."
4. **Language Consensus**: The users liked the ability to use mathematical expressions like π and radians in the tool and the inclusion of files. The language was understandable especially when having a background with using URDF.
5. **Recommendations for Future Evaluations**: It would be more beneficial to conduct the next evaluation at each participant's workplace, as they would be more comfortable with their environment, would likely have access to dual monitors, and could use the web browser to view the robot instead of the integrated view in Visual Studio Code. This would prevent some of the issues participants encountered that hindered their ease of use.

The respondents had positive initial impressions and found various features of the language useful. They provided suggestions for improvements in live graphical feedback, such as highlighting, GUI controls and use of a secondary monitor. The language itself was commonly perceived as intuitive, although each participant held their unique perspective on what constitutes intuitiveness especially with regard to the visualization.

One of the participants expressed a dislike for the transparency of unchanged properties as it hinders comprehensibility, while another participant preferred a more detailed visualization of the changed attributes. This highlights the complexity of designing graphical representations that strike a delicate balance between providing detailed enough information while avoiding clutter that can impede comprehensibility.

The multiple choice response ratings are depicted in the boxplot Fig. 10, which illustrates the variability in the responses. To enhance readability, each question is correspondingly mapped in Table 3.

Table 3. Survey Multiple Choice Questions

Number	Question Text
6	The live graphical feedback helps me to understand what robot properties have changed
7	The live graphical feedback helps me to understand on what robot properties I am working
8	The graphical language helps me to communicate changed robot properties to my colleagues
9	How intuitive did you find the graphical language?
10	How intuitive did you find the live graphical feedback?
11	live graphical feedback would improve my ability to understand and work with robot properties?
12	Does the new workflow improve the communication between colleagues?

The boxplot in Fig. 10 illustrates the distribution of ratings for the multiple-choice questions listed in Table 3. The vertical axis represents the ratings, ranging from positive (5) to negative (1), with neutral responses at (3). The horizontal axis corresponds to the indices of the multiple-choice questions, reflecting the participants' responses.

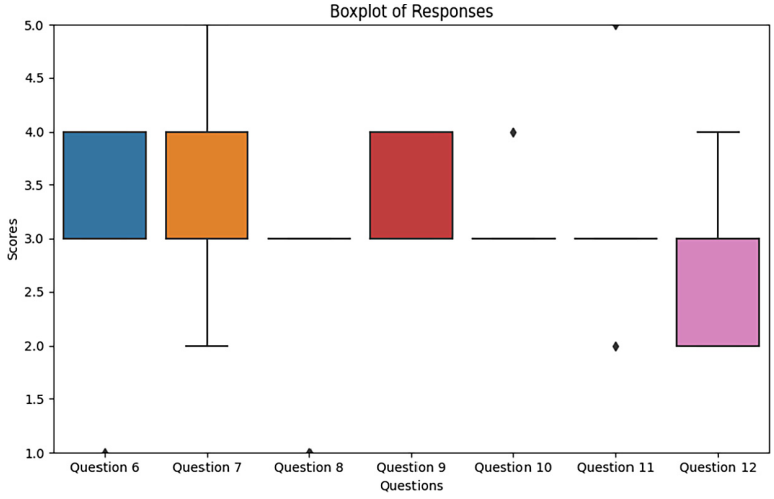

Fig. 10. Boxplot: Questionnaire responses.

Based on the given ratings, we can analyze the reception of the DesignerSL with live graphical feedback among mechatronic engineers for each question:

Question 6: The live graphical feedback helps me to understand what robot properties have changed.

- Mean rating: 3.0
- Standard deviation: 1.22

The engineers' ratings for this question vary, with a mean rating of 3.0.

The standard deviation of 1.22 suggests some level of disagreement or inconsistency among the engineers.

Some engineers may find the live graphical feedback helpful in understanding changes to robot properties, while others may not.

Question 7: The live graphical feedback helps me to understand on what robot properties I am working.

- Mean rating: 3.4
- Standard deviation: 1.14

The engineers, on average, rated the live graphical feedback slightly higher for this question, with a mean rating of 3.4. However, the standard deviation of 1.14 indicates that there is still some variability in their opinions. Some engineers find the live graphical feedback helpful in understanding the robot properties they are working on, while others may not perceive it as useful.

Question 8: The graphical language helps me to communicate changed robot properties to my colleagues.

- Mean rating: 2.6
- Standard deviation: 0.89

The engineers' ratings for this question are relatively low, with a mean rating of 2.6. The low mean rating suggests that the graphical language may not be perceived as effective in communicating changed robot properties to colleagues. The lower standard deviation of 0.89 indicates that the engineers' opinions are more consistent for this question compared to the previous ones.

Question 9: How intuitive did you find the graphical language?

- Mean rating: 3.4
- Standard deviation: 0.55

The engineers, on average, found the graphical language relatively intuitive, with a mean rating of 3.4. The low standard deviation of 0.55 suggests that there is a higher level of agreement among the engineers regarding the intuitiveness of the graphical language.

Question 10: How intuitive did you find the live graphical feedback?

- Mean rating: 3.2
- Standard deviation: 0.45

The engineers rated the live graphical feedback as moderately intuitive, with a mean rating of 3.2. The low standard deviation of 0.45 indicates that there is a

relatively high level of agreement among the engineers regarding the intuitiveness of the live graphical feedback.

Question 11: live graphical feedback would improve my ability to understand and work with robot properties?

- Mean rating: 3.2
- Standard deviation: 1.10

The engineers' opinions are divided for this question, with a mean rating of 3.2 and a higher standard deviation of 1.10. Some engineers believe that live graphical feedback would improve their ability to understand and work with robot properties, while others may not share the same perception.

Question 12: Does the new workflow improve the communication between colleagues?

- Mean rating: 2.8
- Standard deviation: 0.84

The engineers' ratings indicate that the new workflow does not significantly improve communication between colleagues, with a mean rating of 2.8. The standard deviation of 0.84 suggests a moderate level of agreement among the engineers regarding the lack of improvement in communication.

Overall, based on the given ratings, it appears that the mechatronic engineers have mixed opinions about the DesignerSL with live graphical feedback. While they generally find the live graphical feedback and graphical language to be intuitive, their perception of the effectiveness in communicating changed robot properties and improving communication between colleagues is relatively low. There is also some variability in their opinions, as indicated by the standard deviations in the ratings.

The lower scores we observed in colleagues' visual communication of changes (questions 3 and 7) could be attributed to the task sequence. Specifically, task 6, which involves comparing and highlighting robot models, this was only completed by three participants. As not all participants reached this task, as evident in Evalutation Results Table 2 where column six and row participant **'-'** indicate task 6 was not completed, their feedback on this aspect was based on tasks that did not specifically cover this use case. Consequently, these responses may not offer a complete understanding of how the use cases compare and highlight were received by the mechatronic engineers. The limited exposure to these use cases is also reflected in the open questions of the questionnaire.

9 Discussion

In contrast to the existing literature reviewed in Sect. 2, which predominantly concentrates on visually representing properties of robots which can be visually represented such as positions of components and movements, our research places a strong emphasis on highlighting changes that cannot be visually represented, such as inertia and mass.

Leveraging the widely adopted URDF standard tapped into the existing knowledge base of end-users, simplifying their adoption and adaptation to the DSL. This underscores the notion that harnessing established standards can expedite the learning curve for new DSLs. Furthermore, through the incorporation of Xacro, we achieved composability within the language. Moreover, the development of GeometrySL expanded its capabilities with a visual language that empowers hardware engineers to precisely define how the robot is visually represented. This extension encompasses all known keywords and components while eliminating the cumbersome XML syntax, enhancing the user experience.

9.1 Bi-directional Navigation

The benefits of bi-directional interaction, as outlined by Witte et al. [43], is that it reduces cognitive load and improves the overall user experience. By integrating the view and text editor through a bi-directional approach, our tool aims to alleviate cognitive load and further enhance the user experience. It allow users to interact with visual links by clicking on them, instantly directing them to the specific location in the text editor where that element is specified.

Salix has been used in game development to improve collaboration among multiple disciplines [32], resulting in improved collaboration. By leveraging the Salix library, a successful prototype of bi-directional input visualization was achieved. This feature enables users to effortlessly click on visual elements, which then redirects them to the corresponding location within the text editor.

Building upon the insights from [16], our approach embraces the concept of richness by not only providing visual feedback but also integrating the textual and graphical representations. Although persistent changes are not supported via the visualization interface, this integration fosters a cohesive and intuitive user experience by establishing a strong connection between the two interfaces.

9.2 Workflow Comparison

To better understand the benefits of adopting a unified tool, we compare the workflows of hardware engineers without and with the tool. The “Without Tool” workflow represents the current state, where multiple standalone tools and manual processes are used. The “With Tool” workflow demonstrates the envisioned approach using an integrated domain-specific language to streamline tasks. The comparison highlights key differences in efficiency, integration, and user experience, as shown in Table 4.

Table 4. Workflow Comparison: Without Tool vs. With Tool

Aspect	Without Tool	With Tool
Tools and Formats	Relies on various tools, including CAD, Excel, Word, and PowerPoint	Utilizes a unified domain-specific language with integrated functionality
Integration Between Tools	Minimal integration; data is manually transferred between tools	Seamless integration, with CAD files converted via Blender into URDF format for use in the tool
Documentation and Presentation	Separate tools (Word and PowerPoint) are used for documentation and presentations	Unified within the tool, featuring live graphical feedback and customizable views
Change Management	Changes to CAD models are manually documented in Word and presented using PowerPoint	Changes are directly visualized and tracked in the tool, highlighting differences between versions with customized views
User Interaction	Requires manual interaction with multiple tools; data must be consolidated manually	Bi-directional navigation between textual and graphical representations enhances clarity and user experience
Integration with Archives	Separate archiving solutions are required for storage and retrieval	Fully integrated with Philips' existing archiving system for seamless file management
Communication	Disconnected tools and formats may hinder effective interdisciplinary collaboration	A unified domain-specific language and graphical feedback promote better interdisciplinary communication
Efficiency	Workflow is fragmented and complex, leading to inefficiencies	Streamlined and integrated, reducing complexity, enhancing productivity, and improving decision-making

9.3 Immediate Feedback

Earlier research [10] and [14] have indicated that immediate feedback significantly improves debugging. More recent studies also successfully applied immediate feedback in their DSL [3,22,25,31]. Additionally, incorporating immediate feedback ensures that the DSL adheres to the liveness quality, improving the programming experience.

In VS Code, the "summarize event" of the Rascal Language server library is triggered each time the file is saved. We take advantage of this event to re-render the visualization using the latest valid state of the DSL, ensuring an up-to-date representation of the data.

This approach of fully refreshing the visualization, causes a brief moment of disappearance and reappearance of the robot, upon saving the file. This can be improved in terms of user experience. A more gradual change that maintains the robot in the exact same state and keeps it constantly in view would provide a smoother and more user-friendly way of communicating the modifications, minimizing context-switches and enhancing overall usability. By avoiding the robot's disappearance, users can maintain continuous visual feedback and better understand the impact of their changes.

It makes sense to make this part of the summarize behavior the visualization serves a similar purpose as source code errors and warnings, and most languages perform these checks upon saving to achieve it. Additionally, this approach helps minimize the performance impact of running resource-intensive processes with each change.

Our language exhibits key qualities such as liveness, richness, and composability, as discussed in Horowitz et al.'s work on live programming [16].

Like mentioned in Munzner's book [24], we aim to create a solution that addresses the invisibility of property changes and enhances user experience by implementing these highlighting strategies while considering potential conflicts with user-defined materials, overlapping color use, or making components transparent for highlighting. We opted against animations due to their potential to cause change blindness, distracting users from subtle property changes. Instead, we focus on highlighting the changed links to clearly differentiate changes, enhancing user experience by ensuring modifications are easily perceived and understood within the graphical representation. The highlighting differences aim to make changes more distinguishable and improve the user experience.

TMDiff [30] utilizes an algorithm that relies on source location to detect changes, similar to existing tools like Linux's built-in diff and git diff. These tools typically compare lines of code, analyze differences on a line-by-line basis, and categorize changes as modified, deleted, or added.

In contrast, our solution specifically addresses the unique characteristics of URDF and GLS files, which represent robotic systems with distinct links and joints identified by their names. We introduce a novel approach by hashing the links and joints including their unique identifiers while excluding their source location into a hashmap. This allows us to efficiently determine whether links or joints already exist in the system, enabling effective management of modifications and comparisons.

Moreover, our visualization tool offers enhanced capabilities compared to TMDiff. While both systems can highlight changes per link and between linked entities (in the case of joints), our solution goes further by seamlessly accommodating rearrangements of links and joints. This means that even if the structure of the system is altered, our tool can accurately compare the old and modified versions, providing a more robust and flexible comparison mechanism for this use case.

In conclusion, our study has demonstrated that integrating liveness, richness, and composability into a DSL tailored for hardware engineers at Philips IGT, featuring enhancements like bi-directional navigation, live graphical feedback, and the inclusion of robot components, can significantly enhance usability and effectiveness. Furthermore, it has underscored the challenge of visualizing invisible hardware properties, which often hinges on personal preferences and perceptions. These findings make meaningful contributions to the ongoing development of DSLs within this domain, emphasizing the critical role of user feedback in designing intuitive graphical feedback languages.

10 Concluding Remarks

We improved the hardware development workflow at Philips IGT by creating a textual Domain Specific Language (DSL) called GeometrySL or GSL. This DSL was used to formalize handovers from mechanical engineers to mechatronic engineers, preventing mistakes during the exchange process.

The novel approach of leveraging industry-standards URDF and Xacro, alongside techniques such as origin tracing and the LWB Rascal has resulted in a live graphical feedback on differences between Cyber-Physical System (CPS) versions. It enables bi-directional navigation among the graphical representation, URDF and the GSL itself, enhancing the efficiency and usability of the language.

The development of GeometrySL serves as a valuable case study that demonstrates the practical application of immediate, bi-directional visual feedback within an engineering context. The lessons learned from this project can inspire future research efforts and innovations in domain specific languages with live graphical feedback for CPS.

This research mainly focuses on graphically representing textual differences of 3D robot models. In our evaluation of the tool we had diverse feedback. The main challenge we faced was how to visualize changes made to invisible properties such as tolerances, mass inertia.

A potential direction for future work is to address this question and further enhance our research. Specifically, we aim to explore innovative methods for visualizing changes to non-visual properties such as tolerances, mass, and inertia in robotic models. While transparency has been used to some extent, there is an opportunity to investigate alternative approaches that can effectively convey these modifications without significantly altering the robot's overall visual representation. The current side-by-side view, while useful, is limited in its ability to visualize invisible properties, highlighting the need for more advanced visualization techniques.

As we have seen in our research its heavily subjective to what a user perceives as intuitive. Implement options for users to customize the visualization of non-visual properties based on their preferences and specific use cases. This could include adjustable transparency settings, customizable color schemes, or alternative visualization modes tailored to different user needs.

A Appendix

```
data URDF =
    robot(map[str, URDFValue] attributes,
      map[str, list[URDF]] elements)
...
      map[str, list[URDF]] elements) // joint
  | axis(map[str, URDFValue] attributes,
      map[str, list[URDF]] elements) // joint
  | transmission(map[str, URDFValue] attributes,
      map[str, list[URDF]] elements) // robot
  | actuator(map[str, URDFValue] attributes,
      map[str, list[URDF]] elements) // transmission
  | plugin(map[str, URDFValue] attributes,
      map[str, list[URDF]] elements) // robot, link, or  joint
  | counterbalance(map[str,URDFValue] attributes,
      map[str, list[URDF]] elements) // joint
  | tolerance(map[str, URDFValue] attributes,
      map[str, list[URDF]] elements) // robot custom property
  ;
```

Listing 1.14. Fragment of UrdfSL Abstract Data Structure

References

1. Addazi, L., Ciccozzi, F., Langer, P., Posse, E.: Towards seamless hybrid graphical–textual modelling for UML and profiles. In: Anjorin, A., Espinoza, H. (eds.) ECMFA 2017. LNCS, vol. 10376, pp. 20–33. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-61482-3_2
2. Albergo, N., Rathi, V., Ore, J.P.: Understanding xacro misunderstandings. In: 2022 International Conference on Robotics and Automation (ICRA), pp. 6247–6252. IEEE (2022)
3. Alique, D., Linares, M.: The importance of rapid and meaningful feedback on computer-aided graphic expression learning. Educ. Chem. Eng. **27**, 54–60 (2019). https://doi.org/10.1016/j.ece.2019.03.001. https://www.sciencedirect.com/science/article/pii/S1749772818300435
4. Alur, R.: Principles of Cyber-Physical Systems. MIT Press (2015)
5. Ben-Ari, M.: Principles of the Spin Model Checker. Springer (2008)
6. Bolwerk, T.: Improving the workflow for hardware engineers at philips with a domain-specific language and graphical feedback (2023). https://www.cs.ru.nl/masters-theses/2023/T_Bolwerk___Improving_the_workflow_for_hardware_engineers_at_Philips_with_a_domain-specific_language_and_graphical_feedback.pdf
7. Bolwerk, T., Alonso, M., Schuts, M.: Using a textual DSL with live graphical feedback to improve the CPS' design workflow of hardware engineers. In: 2024 19th Conference on Computer Science and Intelligence Systems (FedCSIS), pp. 301–312. IEEE (2024)
8. Bray, T., Paoli, J., Sperberg-McQueen, C.M., Maler, E., Yergeau, F.: Extensible markup language (XML) 1.0 (1998)

9. Cazzola, W., Favalli, L.: Scrambled features for breakfast: concepts of agile language development. Commun. ACM **66**(11), 50–60 (2023)
10. Cook, C., Burnett, M., Boom, D.: A bug's eye view of immediate visual feedback in direct-manipulation programming systems. In: Papers Presented at the Seventh Workshop on Empirical Studies of Programmers, ESP 1997, pp. 20–41. Association for Computing Machinery (1997). https://doi.org/10.1145/266399.266403
11. Cooper, J., Kolovos, D.: Engineering hybrid graphical-textual languages with sirius and xtext: requirements and challenges. In: 2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 322–325 (2019). https://doi.org/10.1109/MODELS-C.2019.00050
12. Fowler, M.: Domain-Specific Languages. Pearson Education (2010)
13. Gray, J., Neema, S., Tolvanen, J.P., Gokhale, A.S., Kelly, S., Sprinkle, J.: Domain-specific modeling. In: Handbook of Dynamic System Modeling, vol. 7, p. 7-1 (2007)
14. Green, T.R.G., Petre, M.: Usability analysis of visual programming environments: a 'cognitive dimensions' framework. J. Vis. Lang. Comput. **7**(2), 131–174 (1996). https://doi.org/10.1006/jvlc.1996.0009. https://www.sciencedirect.com/science/article/pii/S1045926X96900099
15. Hoffmann, B., Urquhart, N., Chalmers, K., Guckert, M.: An empirical evaluation of a novel domain-specific language - modelling vehicle routing problems with athos. Empirical Softw. Eng. **27**(7), 180 (2022). https://doi.org/10.1007/s10664-022-10210-w
16. Horowitz, J., Heer, J.: Live, Rich, and Composable: Qualities for Programming Beyond Static Text (2023)
17. Inostroza, P., van Der Storm, T., Erdweg, S.: Tracing program transformations with string origins. In: International Conference on Theory and Practice of Model Transformations, pp. 154–169. Springer (2014)
18. Kahraman, G., Bilgen, S.: A framework for qualitative assessment of domain-specific languages. Softw. Syst. Model. **14**(4), 1505–1526 (2015). https://doi.org/10.1007/s10270-013-0387-8
19. Klint, P., Van der Storm, T., Vinju, J.: RASCAL: a domain specific language for source code analysis and manipulation. In: Proceedings of the 2009 Ninth IEEE International Working Conference on Source Code Analysis and Manipulation, pp. 168–177. IEEE (2009). https://doi.org/10.1109/SCAM.2009.28
20. Kosar, T., et al.: Comparing general-purpose and domain-specific languages: an empirical study, vol. 7, no. 2, pp. 247–264. https://doiserbia.nb.rs/Article.aspx?ID=1820-02141002247K
21. Kunze, L., Roehm, T., Beetz, M.: Towards semantic robot description languages. In: 2011 IEEE International Conference on Robotics and Automation, pp. 5589–5595 (2011). https://doi.org/10.1109/ICRA.2011.5980170. ISSN 1050-4729
22. Lozano, A., Mens, K., Kellens, A.: Usage contracts: offering immediate feedback on violations of structural source-code regularities, vol. 105, pp. 73–91 (2015). https://doi.org/10.1016/j.scico.2015.01.004. https://www.sciencedirect.com/science/article/pii/S016764231500012X
23. Lyle, J.: Stimulated recall: a report on its use in naturalistic research, vol. 29, no. 6, pp. 861–878. https://www.jstor.org/stable/1502138
24. Munzner, T.: Visualization Analysis and Design. CRC Press (2014). google-Books-ID: NfkYCwAAQBAJ
25. NDC Conferences: Real-time prototyping using visual programming languages - rui martins (2018). https://www.youtube.com/watch?v=cAiFJEcqwm4

26. Onwubolu, G.: Mechatronics: principles and applications. Elsevier (2005)
27. Pérez Andrés, F., de Lara, J., Guerra, E.: Domain specific languages with graphical and textual views. In: Schürr, A., Nagl, M., Zündorf, A. (eds.) AGTIVE 2007. LNCS, vol. 5088, pp. 82–97. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-89020-1_7
28. R. Thorne, D.: Throughput: a simple performance index with desirable characteristics, vol. 38, no. 4, pp. 569–573. https://doi.org/10.3758/BF03193886
29. van Rozen, R., Dormans, J.: Adapting game mechanics with micro-machinations: international conference on the foundations of digital games. Society for the Advancement of the Science of Digital Games (2014)
30. van Rozen, R., van der Storm, T.: Origin tracking + text differencing = textual model differencing. In: Kolovos, D., Wimmer, M. (eds.) ICMT 2015. LNCS, vol. 9152, pp. 18–33. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-21155-8_2
31. van Rozen, R.: Cascade: a meta-language for change, cause and effect (2022). https://ir.cwi.nl/pub/32568
32. van Rozen, R., Reijne, Y., Julia, C., Samaritaki, G.: First-person realtime collaborative metaprogramming adventures (2021). https://ir.cwi.nl/pub/31301/
33. van Rozen, R., van der Storm, T.: Toward live domain-specific languages: from text differencing to adapting models at run time, vol. 18, no. 1, pp. 195–212 (2019). https://doi.org/10.1007/s10270-017-0608-7
34. Schuts, M., Aarssen, R., Tielemans, P., Vinju, J.: Large-scale semi-automated migration of legacy C/C++ test code. Softw. Pract. Exp. **52**(7), 1543–1580 (2022)
35. Schuts, M., Alonso, M., Hooman, J.: Industrial experiences with the evolution of a DSL. In: Proceedings of the 18th ACM SIGPLAN International Workshop on Domain-Specific Modeling, pp. 21–30 (2021)
36. Shelly, G.B., Vermaat, M.E.: Microsoft Office 2010: Introductory. Course Technology Press (2012)
37. Shen, L., Chen, X., Liu, R., Wang, H., Ji, G.: Domain-specific language techniques for visual computing: a comprehensive study, vol. 28, no. 4, pp. 3113–3134 (2021). https://doi.org/10.1007/s11831-020-09492-4
38. Shih, R.H.: Parametric Modeling with Creo Parametric 2.0. SDC Publications (2013)
39. Steenvoorden, T., Stutterheim, J., Barendsen, E., Plasmeijer, R.: Visual support for learning monads, pp. 130–139. Global Science and Technology Forum. https://doi.org/10.5176/2251-2195_CSEIT17.64
40. van der Storm, T.: Semantics engineering with concrete syntax. In: Eelco Visser Commemorative Symposium (EVCS 2023). Schloss Dagstuhl-Leibniz-Zentrum für Informatik (2023)
41. van Rozen: Live game programming with micro-machinations and rascal (2014). https://www.youtube.com/watch?v=YzsKaJEX4D4
42. Vandierendonck, A.: A comparison of methods to combine speed and accuracy measures of performance: a rejoinder on the binning procedure, vol. 49, no. 2, pp. 653–673. https://doi.org/10.3758/s13428-016-0721-5
43. Witte, T., Tichy, M.: A hybrid editor for fast robot mission prototyping. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW), pp. 41–44 (2019). https://doi.org/10.1109/ASEW.2019.00026. ISSN 2151-0830

44. Woltz, D.J., Was, C.A.: Availability of related long-term memory during and after attention focus in working memory, vol. 34, no. 3, pp. 668–684. https://doi.org/10.3758/BF03193587
45. Xue, D., Chen, Y.: System simulation techniques with MATLAB and Simulink. Wiley (2013)